

Học sâu 2026: Biểu diễn Dữ liệu bằng Tensor

Nền tảng Toán học của Mạng Nơ-ron (Dựa trên Keras & NumPy)



Tensor là gì?

Tensor là một container chứa dữ liệu, thường là dữ liệu số. Đây là cấu trúc dữ liệu nền tảng cho mọi hệ thống Machine Learning hiện đại.

Toán học vs. Lập trình

Cách mô tả toán học rõ ràng và không mơ hồ nhất chính là mã nguồn thực thi (executable code).



Scalars (Rank-0 Tensors)

- **Định nghĩa:** Tensor chỉ chứa MỘT con số duy nhất.
- **Đặc điểm:** Số trục (axes / ndim) = 0.
- **Ví dụ:** Kiểu số thực float32 hoặc float64.

```
import numpy as np  
  
x = np.array(12)  
x.ndim # Output: 0
```

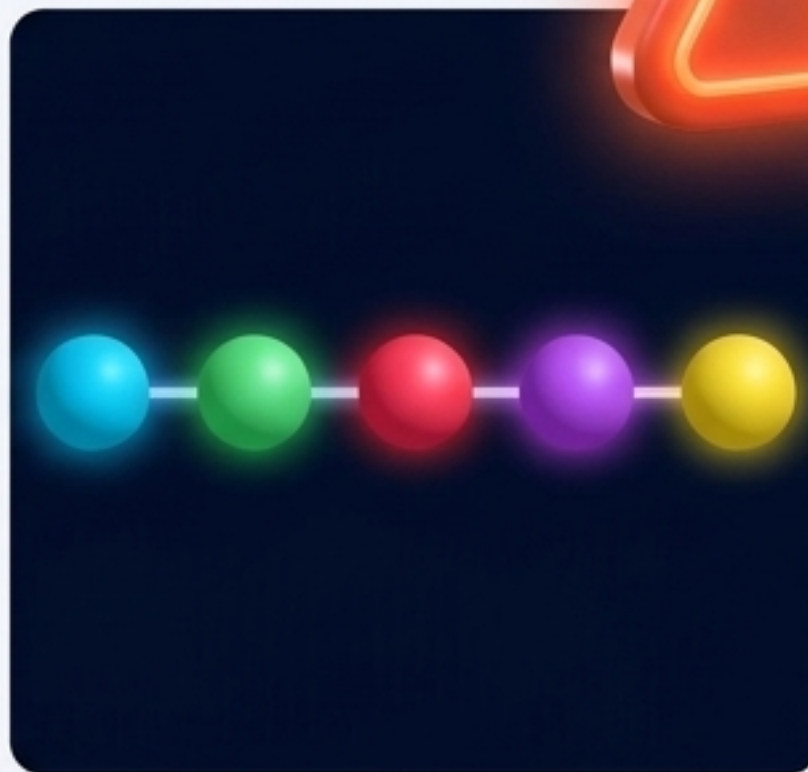


Vectors (Rank-1) & Cạm bẫy Khái niệm

Mảng các con số, sở hữu đúng 1 trục (axis).

```
x = np.array([12, 3, 6, 14, 7])  
x.ndim # Output: 1 (Shape: (5,))
```

Vector 5 chiều



Vector 5 chiều: 1 trục duy nhất chứa 5 phần tử.

Tensor 5 chiều



Tensor 5 chiều: Có 5 trục tọa độ không gian độc lập.

Đừng nhầm lẫn giữa số lượng phần tử trên một trục và số lượng trục.

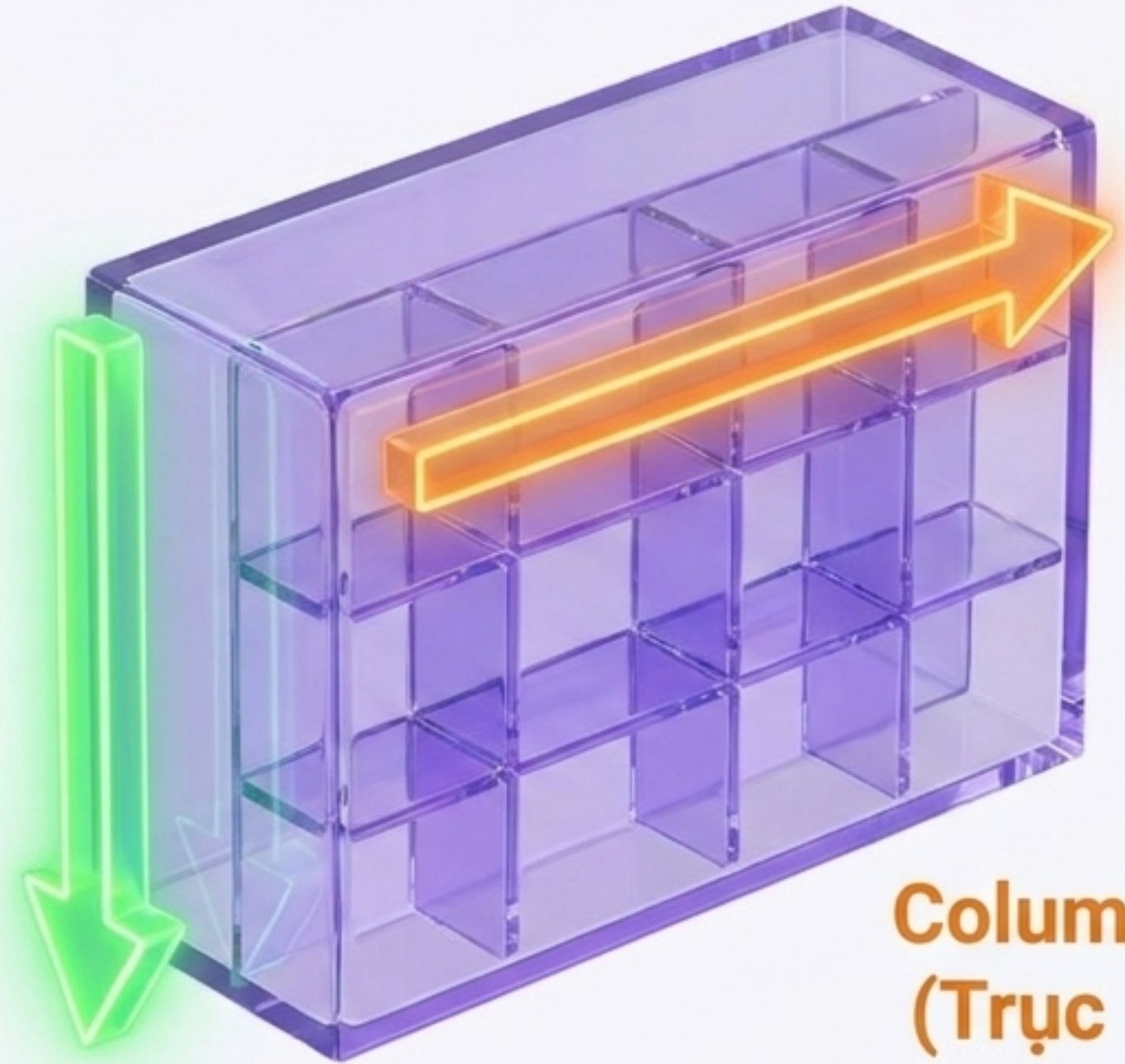
Matrices (Rank-2 Tensors)

Mảng của các vectors. Biểu diễn dữ liệu dưới dạng lưới số chữ nhật.

```
x = np.array([[5, 78, 2],  
              [6, 79, 3]])
```

```
x.ndim # Output: 2
```

Rows
(Trục 0)



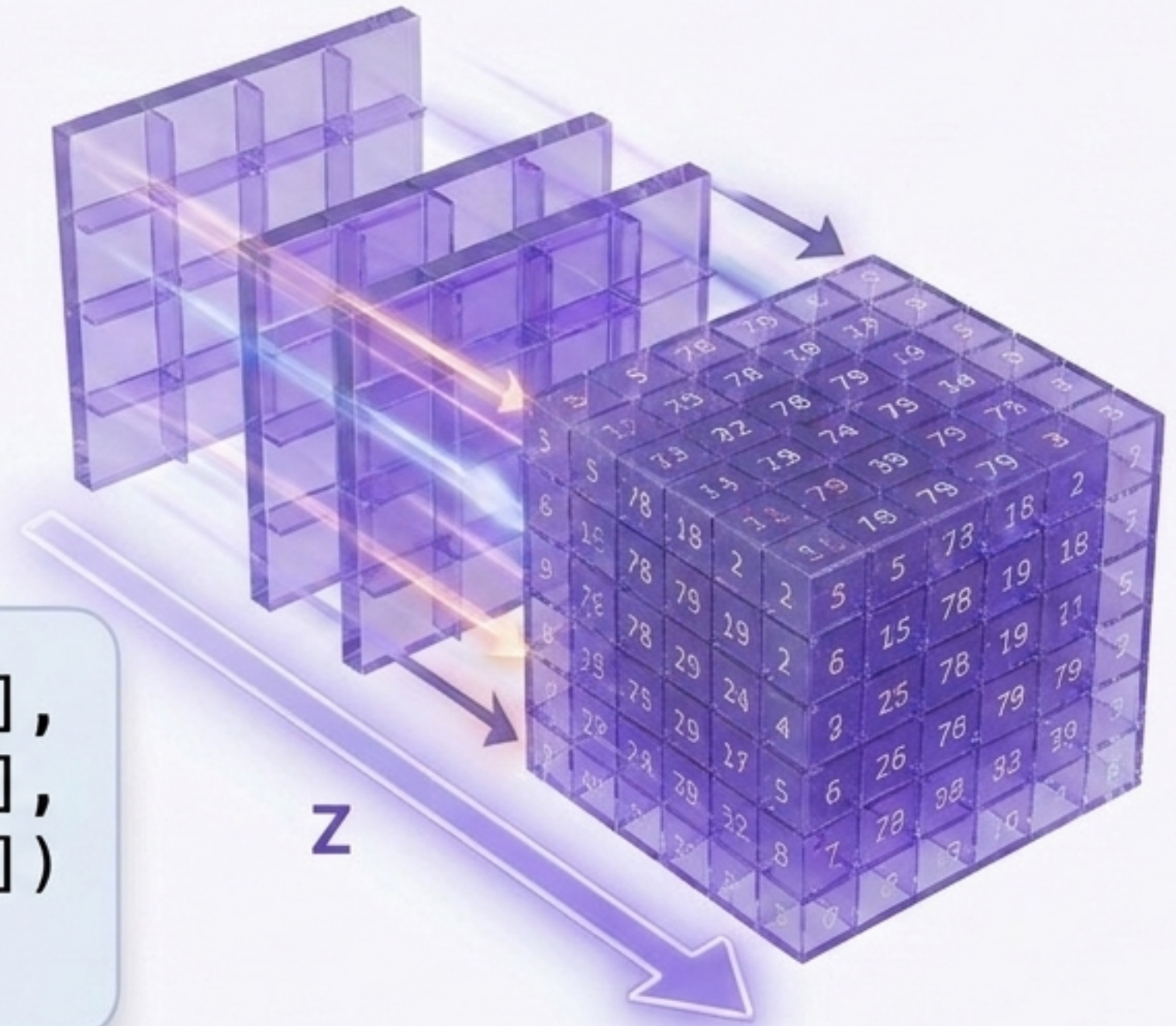
Columns
(Trục 1)

Rank-3 & Higher-rank Tensors

Quy luật Xếp chồng: Đóng gói Matrices -> Rank-3 (Khối hộp). Đóng gói Rank-3 -> Rank-4...

Thực tiễn Deep Learning: Thường xử lý các tensor từ Rank-0 đến Rank-4 (Rank-5 cho video).

```
x = np.array([[5, 78, 2], [6, 79, 3]],  
             [[5, 78, 2], [6, 79, 3]],  
             [[5, 78, 2], [6, 79, 3]])  
x.ndim # Output: 3
```



3 Thuộc tính Cốt lõi Xác định Tensor

Number of axes (ndim)

Hạng (Rank) của tensor.

Ví dụ MNIST:

`train_images.ndim == 3`

Shape

Tuple mô tả kích thước dọc theo từng trục.

Ví dụ MNIST:

`train_images.shape == (60000, 28, 28)`

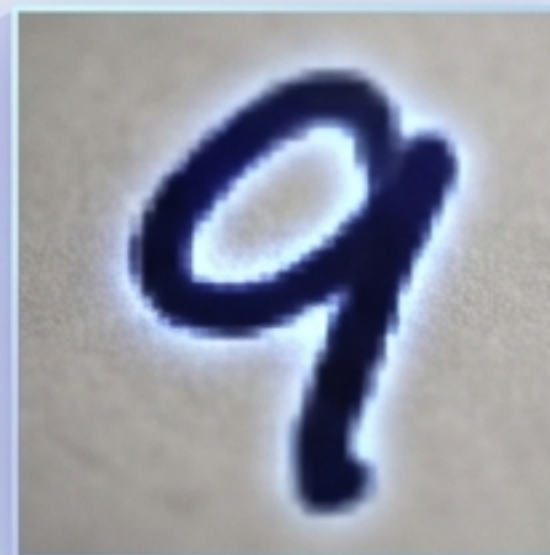
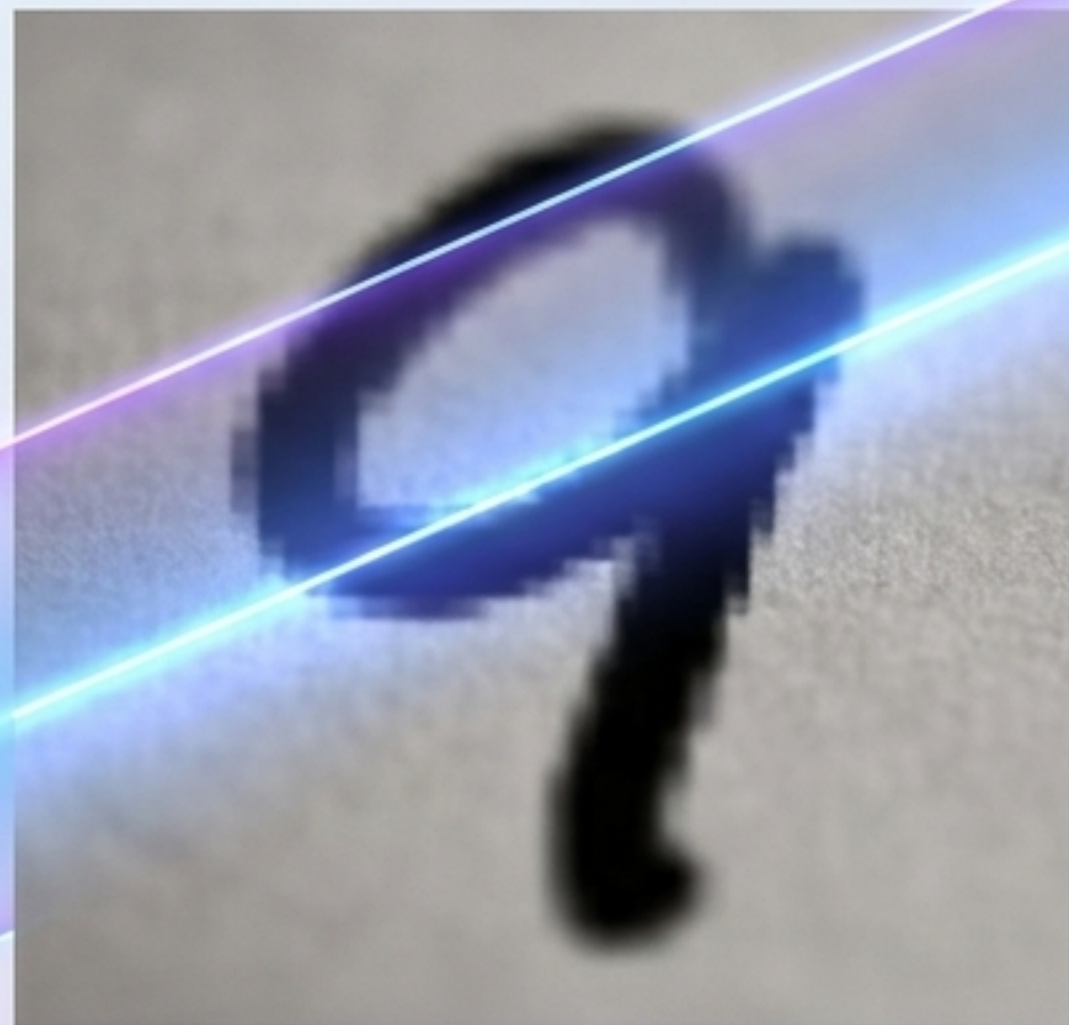
Data type (dtype)

Kiểu dữ liệu lưu trữ (float32, uint8, ...).

Ví dụ MNIST: `train_images.dtype == uint8`

Cắt lát Tensor (Tensor Slicing)

Kỹ thuật trích xuất dữ liệu con (subset) bằng cú pháp mảng NumPy.



Chọn 90 ảnh (từ index 10 đến 99):
`my_slice = train_images[10:100] -> Shape: (90, 28, 28)`

Cắt góc phải dưới (14x14 pixels) của mọi ảnh:
`my_slice = train_images[:, 14:, 14:]`

Crop trung tâm (dùng chỉ số âm):
`my_slice = train_images[:, 7:-7, 7:-7]`

Trục Batch (Batch Axis) - Tại sao phải chia lô?



- **Trục Mẫu (Samples axis):** Trục 0 luôn đại diện cho các mẫu dữ liệu.
- **Giới hạn Phần cứng:** Mô hình không thể nạp toàn bộ dataset (ví dụ 60,000 ảnh) vào bộ nhớ (RAM/VRAM) cùng một lúc.
- **Giải pháp:** Chia nhỏ dữ liệu thành các lô (Batches). Khi đánh giá theo lô, Trục 0 được gọi là **Trục Batch (Batch axis)**.

```
batch_1 = train_images[:128] # Lô 1: Shape (128, 28, 28)
batch_2 = train_images[:128] # Lô 2: Shape (128, 28, 28)
batch_n = train_images[128:256] # Lô 2
batch_n = train_images[128*n : 128*(n+1)]
```

Tensor Thực tế: Dữ liệu dạng Vector (Rank-2)

Shape: (samples, features)



Examples Panel

1



Dữ liệu Bảo hiểm

100,000 người. Mỗi người có 3 đặc trưng (tuổi, giới tính, thu nhập).

Shape: (100000, 3)

2



Dữ liệu Đếm từ (Text)

500 văn bản. Từ điển đếm tần suất có 20,000 từ.

Shape: (500, 20000)

Tensor Thực tế: Chuỗi Thời gian (Rank-3)

Shape: (samples, timesteps, features)

Examples Panel

Dữ liệu Chứng khoán

250 ngày giao dịch (samples). Mỗi ngày ghi nhận 390 phút (timesteps). Mỗi phút lưu 3 mức giá: hiện tại, cao nhất, thấp nhất (features).

Shape: (250, 390, 3)

Timesteps

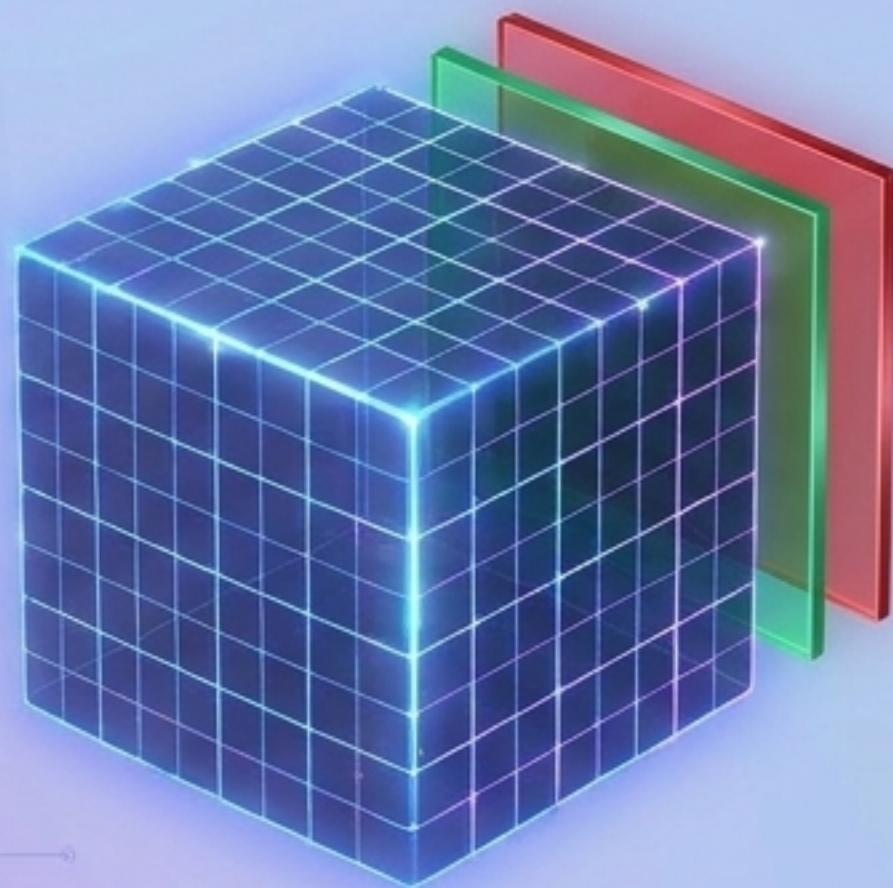
Khi **thời gian** (hoặc **thứ tự chuỗi**) quan trọng, dữ liệu được lưu bằng Rank-3 Tensor.
Quy ước: Trục thời gian luôn là Trục 1.

Tensor Thực tế: Dữ liệu Hình ảnh (Rank-4)

Hình ảnh yêu cầu 3 chiều (Cao, Rộng, Kênh màu). Có 2 quy ước sắp xếp trục gây tranh cãi giữa các Framework.

Channels-last (TensorFlow/Keras)

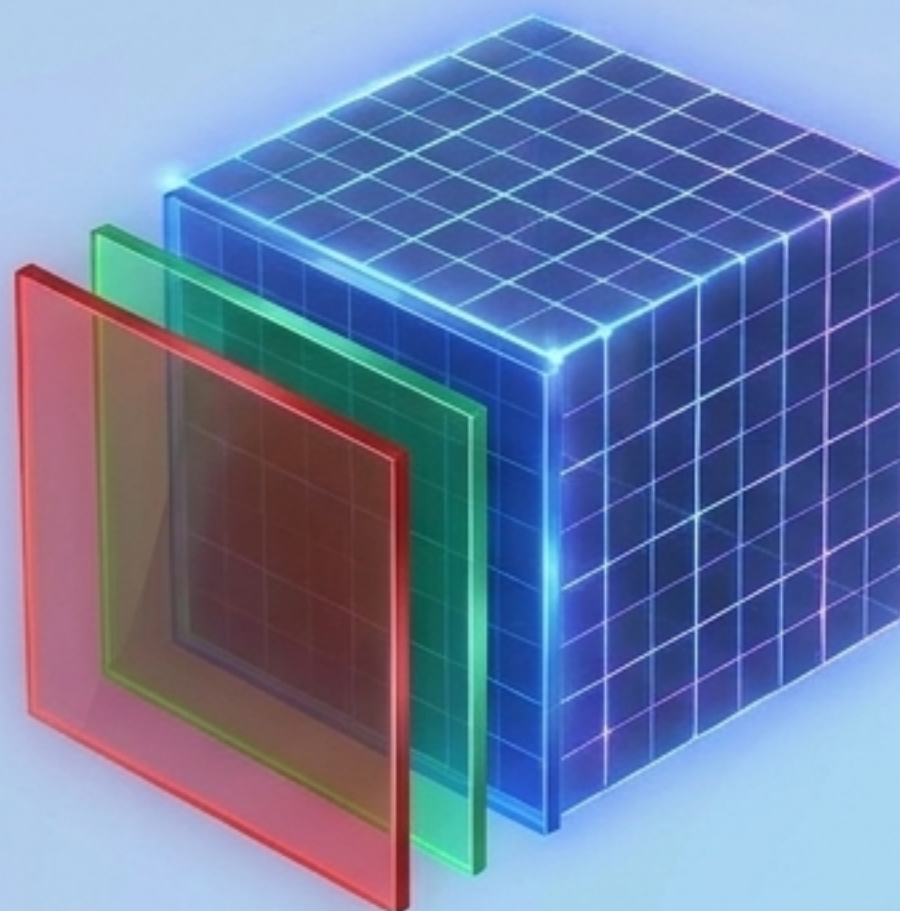
Shape: (samples, height, width, color_depth)



Channels-first (PyTorch)

Shape: (samples, color_depth, height, width)

 PyTorch



Tensor Thực tế: Dữ liệu Video (Rank-5)

Shape: (samples, frames, height, width, channels)

Phân tích Độ nặng Dữ liệu

Kịch bản: Batch gồm 4 video YouTube. Mỗi video dài 60s, 4 fps (240 frames), độ phân giải 144x256, 3 kênh màu RGB.

Toán học: $4 * 240 * 144 * 256 * 3 = 106,168,320$ giá trị.

**Thực tế: Nếu dùng float32 (32 bit/giá trị)
-> Nặng 425 MB cho một lô siêu nhỏ!**

Kết luận: Giải thích lý do tại sao Video trong đời thực bắt buộc phải nén (như MPEG) và không lưu dưới dạng float32.

